SIMATS ENGINEERING
CO5 ASSESSMENT TOOL 3

NAME : JAIVANT.S
REG NO : 192311326
COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS
COURSE CODE : CSA0613

CO5 – AT3 DAA Mock Interview

Interviewer: Welcome to the CO5 – AT3 DAA Mock Interview. This interview will take approximately 15 minutes. Please introduce yourself briefly. Once you are ready, we will begin.

Student:

Good morning, sir. My name is Jaivant. I am a final-year Computer Science Engineering student. I have a good interest in algorithms and problem-solving, especially topics like Dynamic Programming, Backtracking, Greedy Algorithms, and graph algorithms. I have learned the basic concepts of algorithm design and complexity analysis, and I am looking forward to applying those concepts to solve problems. I'm ready for the interview.

Question 1 — Backtracking

Interviewer: What is backtracking? Can you explain it with an example?

Student:

Backtracking is a problem-solving technique where we build a solution step by step and check whether the current choice satisfies the given constraints. If the choice does not lead to a valid solution, we go back, or backtrack, and try another choice.

A common example is the **N-Queens problem**. We place queens one by one on the chessboard. After placing each queen, we check whether it conflicts with the previously placed queens. If there is a conflict, we remove the queen and try another position.

So, backtracking mainly uses recursion, constraint checking, and pruning to avoid exploring unnecessary possibilities.

Question 2 — State-Space Tree

Interviewer: What is a state-space tree, and why is it useful in backtracking?

Student:

A state-space tree is a tree representation of all possible choices or states of a problem. Each node represents a partial solution, and each branch represents a possible decision.

It is useful in backtracking because it allows us to systematically explore the solution space. If a particular node violates a constraint, we don't need to explore its children, so that entire branch can be pruned.

For example, in N-Queens, each level of the tree can represent placing one queen, and each branch represents a possible column for that queen. Invalid placements can be eliminated immediately.

Question 3 — Pruning

Interviewer: What is pruning? Why is it important in backtracking algorithms?

Student:

Pruning means eliminating a branch of the state-space tree when we know that it cannot produce a valid solution.

It is important because without pruning, backtracking may have to explore a very large number of possible combinations. By checking constraints early, we can avoid unnecessary computation.

For example, in the subset sum problem, if the current sum already exceeds the target, we can stop exploring that branch when all remaining values are non-negative. This reduces the search space and improves practical performance, although the worst-case complexity may still remain exponential.

Question 4 — N-Queens

Interviewer: How would you solve the N-Queens problem using backtracking?

Student:

I would place one queen in each row and recursively try every possible column.

Before placing a queen, I would check three conditions: whether another queen is already in the same column, whether there is a queen on the same left diagonal, and whether there is a queen on the same right diagonal.

If the position is safe, I place the queen and recursively move to the next row. If no position works for the current row, I remove the previously placed queen and backtrack.

The process continues until all N queens are successfully placed. If all rows are filled, we have found a solution.

The worst-case time complexity is exponential, commonly expressed as approximately **O(N!)**, while the space used by the recursion and board representation is **O(N)** or **$O(N^2)$** depending on the implementation.

Question 5 — Subset Sum

Interviewer: Suppose you are given a set of positive integers and a target value. How can backtracking determine whether a subset with exactly that sum exists?

Student:

I would process the elements one by one and make two choices for every element: either include it in the subset or exclude it.

At each stage, I maintain the current sum. If the current sum becomes equal to the target, I return true. If the sum exceeds the target, I can prune that branch because all the numbers are positive. Then I recursively continue with the next element. If one branch fails, I explore the other branch. Since every element has two possible choices, the worst-case time complexity is **$O(2^n)$**, where n is the number of elements. The recursion requires **O(n)** space.

Question 6 — Hamiltonian Cycle

Interviewer: What is a Hamiltonian Cycle, and how can backtracking be used to find one?

Student:

A Hamiltonian Cycle is a cycle in a graph that visits every vertex exactly once and finally returns to the starting vertex.

Using backtracking, I start with one vertex and build the path one vertex at a time. For every new vertex, I check whether it is adjacent to the previous vertex and whether it has already been included in the path.

If the choice is valid, I add the vertex and continue recursively. If no valid vertex can be added, I backtrack and try another vertex.

When all vertices have been included, I additionally check whether the last vertex is connected to the starting vertex. If it is, we have found a Hamiltonian Cycle.

Question 7 — Graph Coloring

Interviewer: Explain the graph coloring problem using backtracking.

Student:

The graph coloring problem is to assign colors to the vertices of a graph such that no two adjacent vertices have the same color, usually using at most a given number of colors.

Using backtracking, I assign a color to one vertex at a time. For each vertex, I try every available color and check whether that color conflicts with any already colored adjacent vertex.

If the color is valid, I recursively color the next vertex. If no color is possible, I remove the previous assignment and try another color.

The backtracking approach explores different color assignments while pruning assignments that violate the adjacency constraint.

Question 8 — P, NP, NP-Hard and NP-Complete

Interviewer: Can you differentiate between P, NP, NP-Hard, and NP-Complete problems?

Student:

Yes, sir.

P contains problems that can be solved in polynomial time by a deterministic algorithm.

NP contains decision problems where a proposed solution can be verified in polynomial time.

NP-Hard problems are at least as difficult as every problem in NP. They do not necessarily have to belong to NP or even be decision problems.

NP-Complete problems are both in NP and NP-Hard.

For example, problems such as **Hamiltonian Cycle, Clique, and 3-SAT** are NP-Complete.

An important point is that if any NP-Complete problem is solved in polynomial time, then all problems in NP can also be solved in polynomial time, which would imply **P = NP**.

Question 9 — Greedy vs Backtracking

Interviewer: What is the difference between a Greedy algorithm and a Backtracking algorithm?

Student:

A Greedy algorithm makes the best-looking choice at the current step and normally does not reconsider that choice. It works efficiently when the problem has the greedy-choice property.

Backtracking, on the other hand, explores different possibilities and goes back when a choice leads to an invalid or unsuccessful solution.

For example, activity selection can be solved using a Greedy approach because choosing the activity with the earliest finishing time leads to an optimal solution.

N-Queens is a typical Backtracking problem because a locally valid queen placement may later lead to failure, so we need to undo choices and try alternatives.

So, the main difference is that **Greedy commits to local choices, while Backtracking explores and reverses choices when necessary.**

Question 10 — Scenario-Based

Interviewer: Suppose you are given a very large problem where an exact backtracking solution takes exponential time. What would you do to make the solution practical?

Student:

First, I would look for opportunities to reduce the search space using better pruning and bounding functions. I would also check whether there are duplicate or symmetric states that can be avoided.

Then I would analyze whether the problem has properties that allow Dynamic Programming, Greedy algorithms, or polynomial-time solutions instead of pure backtracking.

If an exact solution is still computationally expensive and the application allows approximate answers, I would consider an approximation or heuristic algorithm.

So, I would first try to improve the exact algorithm using pruning and optimization, and if the problem is still too large, I would consider a suitable approximate or heuristic approach based on the application's requirements.

Interview Completed

Question-wise Performance

Question	Performance
Q1. Backtracking	Strong understanding
Q2. State-Space Tree	Correct and clear
Q3. Pruning	Strong
Q4. N-Queens	Strong
Q5. Subset Sum	Strong
Q6. Hamiltonian Cycle	Good
Q7. Graph Coloring	Good
Q8. P, NP, NP-Hard, NP-Complete	Strong
Q9. Greedy vs Backtracking	Strong

Question	Performance
Q10. Scenario-based	Strong

Technical Strengths

- Good understanding of **Backtracking and recursion**
- Correct understanding of **state-space trees and pruning**
- Strong knowledge of **N-Queens and Subset Sum**
- Clear understanding of **NP-Complete concepts**
- Able to compare **Greedy and Backtracking**
- Good awareness of **complexity and scalability**

Areas Requiring Improvement

- Could explain recurrence relations more formally.
- Could improve precision when discussing space complexity for different implementations.
- Should practice drawing state-space trees and explaining them verbally.
- Could strengthen knowledge of **polynomial-time reductions and approximation algorithms**.

Final Evaluation

Criterion	Level	Marks
Technical Knowledge	Level 4	23/25
Problem Solving	Level 4	23/25
Analytical Thinking	Level 4	14/15
Communication	Level 4	14/15
Confidence	Level 4	9/10
Response to Questions	Level 4	9/10
Total		**92/100**

Overall Performance: Excellent — 92/100

The performance demonstrates a **strong understanding of DAA concepts**, particularly Backtracking, pruning, state-space trees, and complexity analysis. The main focus for improvement should be on giving more precise technical explanations and being comfortable with reduction proofs and advanced optimization techniques.